

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Taller de Diseño Digital (CE3201)

Laboratorio 2 – Propuesta de diseño

Lógica Combinacional y Aritmética

Segundo Semestre

Profesor:

Luis Alonso Barboza Artavia

Estudiante:

Mauro Andrés Navarro Obando

Yonathan Monge Sanabria

Grupo 01

Fecha de entrega: miércoles 03 de setiembre

Año 2025

Enunciado del problema a desarrollar

3.1. Problema 1

Se deberá realizar una calculadora parametrizable, con el fin de que pueda ejecutar las siguientes operaciones: suma, resta, multiplicación, división, módulo, and, or, xor, shift left y shift right. Se debe de implementar las banderas de estados de la ALU: Negativo (N), Cero (Z), Acarreo (C) y Desbordamiento (V). En la figura 1 se muestra el diagrama de primer nivel. Con base en la descripción anterior:

Propuestas de Diseño

Propuesta 1:

Circuitos básicos para operaciones elegidas

- Suma:

Para lograr hacer una suma completa de N bits, se propone el uso de un Full-Adder, su funcionamiento se basa en la siguiente tabla de verdad:

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	1	1
0	1	1	1	0
1	0	0	1	1
1	0	1	1	0
1	1	0	0	0
1	1	1	0	1

Dónde A, B son los bits de entrada que se suman, C_{in} es el bit del posible carry de la suma anterior, por otra parte, S representa la salida resultante de la suma entre A y B, C_{out} es el bit carry resultante de la suma entre A y B.

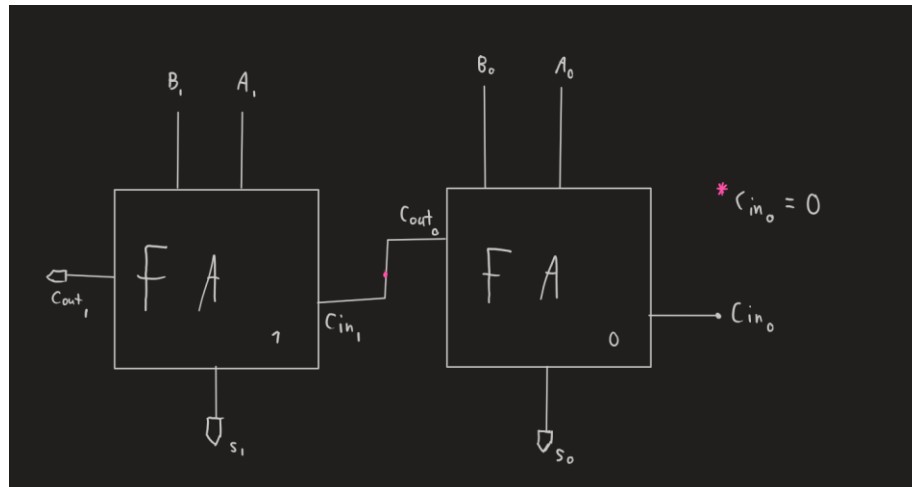
La ecuación booleana del comportamiento para la salida del bit S es el siguiente

$$S = (A \oplus B) \oplus C_{in}$$

La ecuación booleana del comportamiento para la salida del bit C_{out} es el siguiente:

$$C_{out} = (A \text{ or } B) \text{ or } [(A \oplus B) \text{ and } C_{in}]$$

El siguiente diagrama ejemplifica la conexión de los Full-Adder:

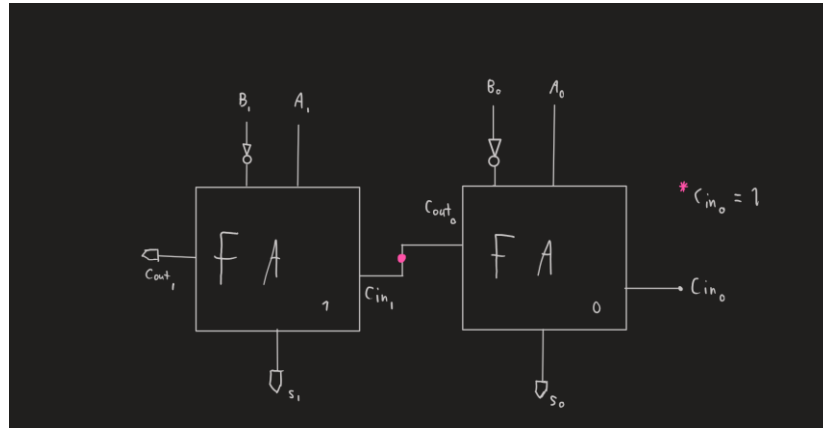


- Resta:

En el caso de la resta se realiza un ajuste al sumador completo anterior (Full-Adder), dicho ajuste se trata de negar todos los bits de entrada del número B e inicializar el bit menos significativo del carry con un valor de 1 ($C_{in} = 1$), se debe aclarar que el B representa el número que sustrae (sustraendo), esto permite sumarle un bit al complemento del número B, la tabla de verdad del funcionamiento del restador completo es de la siguiente forma:

A	B	C_{in}	C_{out}	S
0	0	0	1	1
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	0

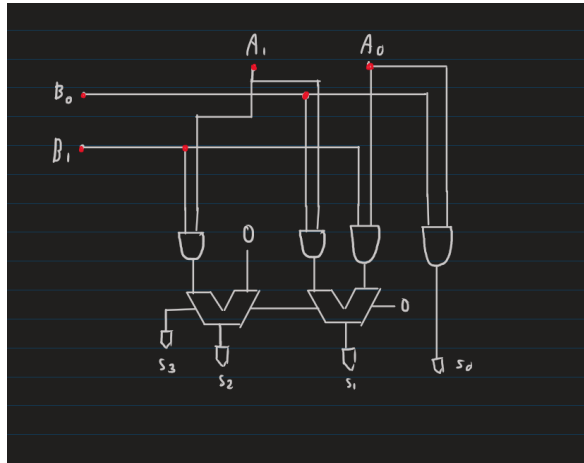
Diagrama del restador completo:



- Multiplicación:

Para terminar, como posible solución se toma en cuenta el diseño llamado “multiplicador array” el cual consiste en un arreglo de compuertas AND los cuales calculan los productos parciales y una red de sumadores que suman los dichos productos, este diseño se encuentra en el libro “Sara Harris, David Harris”, el diagrama de la segunda imagen muestra los componentes y arquitectura necesaria para que se realice de forma correcta la multiplicación de un número A y B como en el ejemplo de la primer imagen:

$$\begin{array}{r}
 \begin{array}{cc} A_1 & A_0 \\ | & 0 \end{array} \quad (2) \\
 \times \begin{array}{cc} B_1 & B_0 \\ | & 1 \end{array} \quad (3) \\
 \hline
 \begin{array}{ccc} & 1 & 0 \\ + & 1 & 0 & - \\ \hline
 1 & 1 & 0 \end{array} \quad (6)
 \end{array}$$

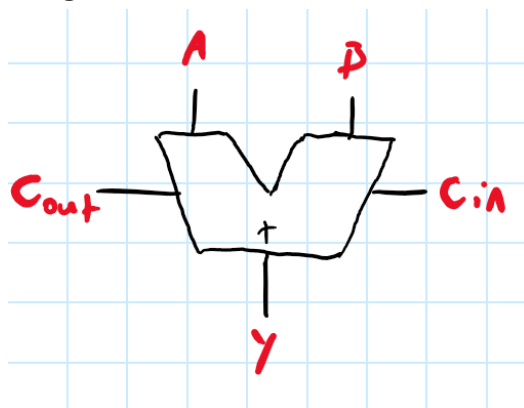


Propuesta 2:

Circuitos básicos para operaciones elegidas

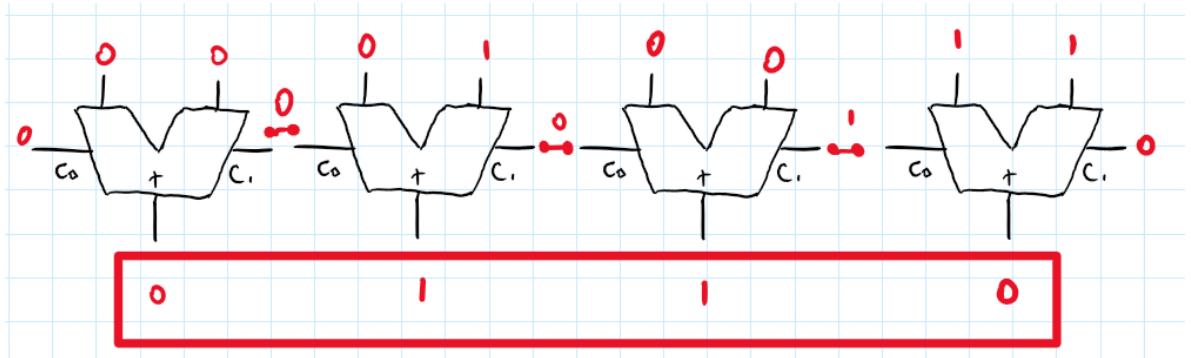
- Suma:
Según se comenta en el libro Harris & Harris las sumas se realizan por medio de un CPA (Carry Propagate Adder) dentro de esta categoría tenemos varias maneras de diseñar sumadores. El Sumador más general que puede hacerse parametrizable es el llamado Ripple-Carry Adder, el cual consiste en encadenar N cantidad de CPA full adders.

Imagen de un full adder:



Cin es el Carry de entrada, cada compuerta tiene dos bits de entrada A y B y puede enviar un carry al siguiente par de números binarios y entrega u

resultado Y. Al colocar varios en “serie” tomando la conexión entre ellos como C_{in} y C_{out} podemos formar le siguiente sumador:



El Sumador realiza la siguiente operación, comprobando que el arreglo de full-adders es equivalente a realizar la operación de bits:

$$\begin{array}{r}
 0001 \\
 + 0101 \\
 \hline
 0110
 \end{array}$$

Tabla de verdad de los full-adders:

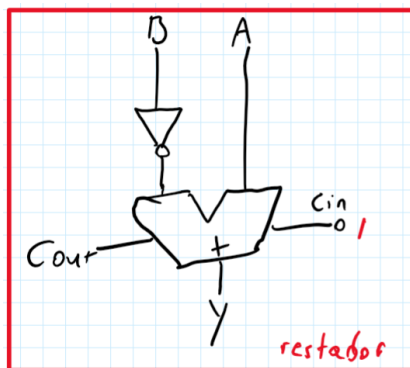
Entradas: (C_{in} , A y B)

Salidas: (C_{out} y Y)

C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- Resta:

La resta es muy similar a la suma pero lo que se hace es aplicar doble complemento al número que está en el minuendo, y luego se procede a hacer una suma normal. Para aplicar el doble complemento se niega la entrada del minuendo y se le suma uno. Luego se pasa por un CPA como el de la suma.



Donde:

$$A - B = Y$$

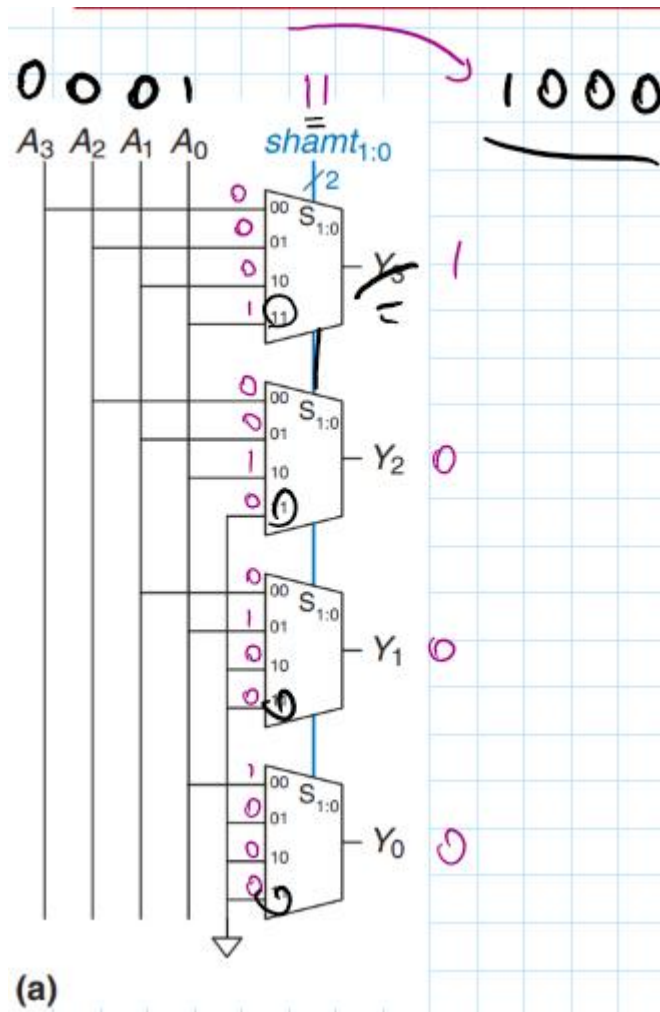
Handwritten binary subtraction showing the result of $0101 - 0011$ (which is $5 - 3$ in decimal). The result is 10010 (decimal 2). The intermediate steps are: $0101 - 0011 = 1100$, then $1100 - 1100 = 0000$, and finally 10010 (with a red bracket under the last three digits labeled 2_{10}).

- Multiplicación:

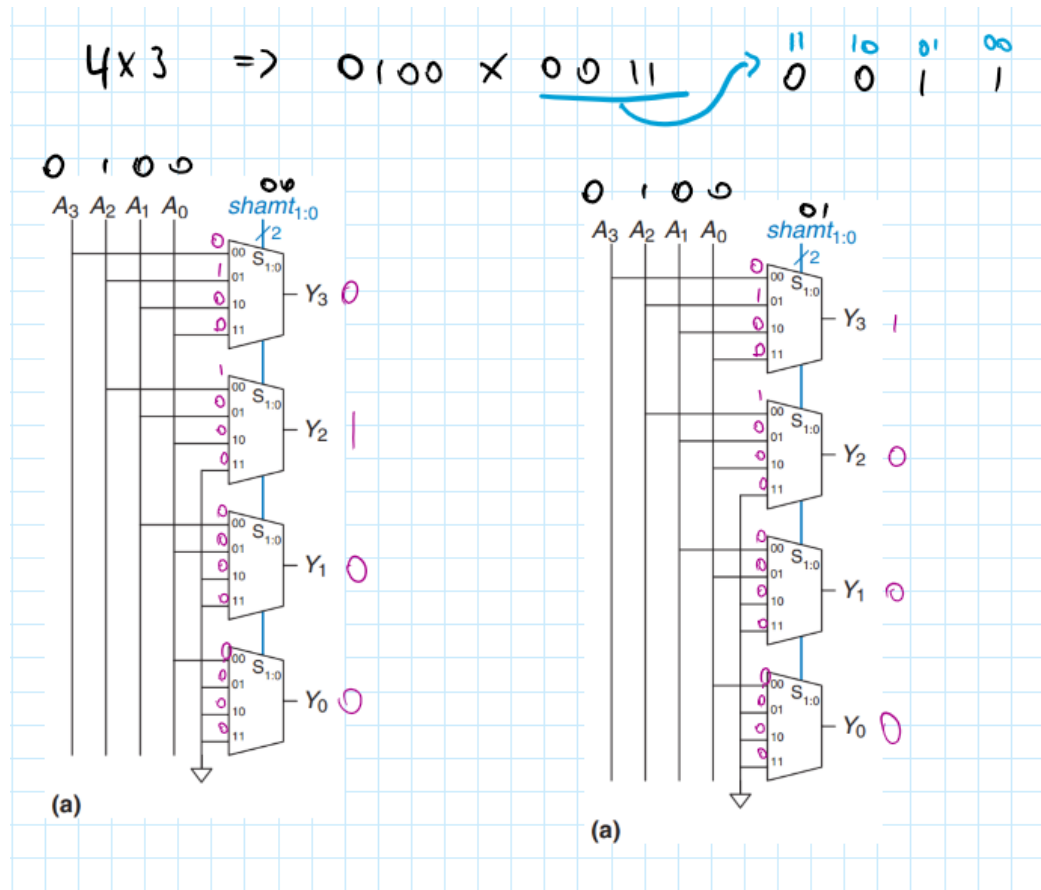
Para la multiplicación se ha tomado la funcionalidad de la operación (shift-left) que funciona para multiplicar un número por **alguna potencia de 2**. La cantidad de movimientos a la izquierda (shift-left) que se apliquen a un número binario es la equivalente a: el número (2^n movimientos)

Handwritten binary shift operation: $0011 \ll 2 = 3 \cdot 2^2 = 12$. The result is $1100 = 12 \equiv V_0$.

Un left-shifter se puede aplicar por medio de multiplexores, de modo que un left-shifter que maneje números de 4-bits se verá de la siguiente manera:



Aplicando shifters y sumas, podemos multiplicar 2 cantidades como se muestra en el siguiente ejemplo:



En la imagen podemos observar cómo se intenta realizar la operación 4 x3 la cual es igual a 12 y en binario la operación se ve como

$$0100 \times 0011 = 1100$$

El 3 se descompone en potencias de 2 como $2^1 + 2^0$ lo que es posteriormente representado al convertir el 3 a binario en el propio número 0011. Se le da un ID a cada bit del número 0011 y los que quedan encendidos en 1 son las entradas de los multiplexores usan para determinar cuántos shift-left realizar al número original. Finalmente se suman los resultados y se verifica que la suma es correcta:

$$\begin{array}{r} 0100 \\ 1000 \\ \hline 1100 = 12 \end{array}$$

Propuesta elegida: Suma y resta de propuesta 1 y multiplicación de la propuesta 2.

Justificación: Para suma y resta ambas propuestas fueron relativamente iguales, aunque se desarrolló mejor la idea de la primera propuesta.

Para multiplicación la de la propuesta dos fue más comprensible y parametrizable.